

Computer Vision tasks with SPARK

Use a multimodal LLM to describe an image, then detect and draw bounding boxes around specific objects (slides 21-23).

Activity is designed by Su Nguyen; Detailed guide is drafted by Claude Cowork.

Before you start

You need a Google AI (Gemini) API key - the same one from the Week 5 SPARK activity (Google AI Studio). Open Settings (gear, top-right), choose Google Gemini as the AI Provider, paste your key, and click Save Settings before running any flow. Never share or screenshot your key. The image used here (arsenal-football.jpg) is in the Week 6 > data > images folder.

Tool	SPARK - Studio for Prototyping Agent Reasoning & Knowledge
Model	Google Gemini (gemini-2.5-flash)
Input	arsenal-football.jpg - a crowded football match scene
What you build	Image Descriptor (Upload Image -> Image Vision -> Show Description), then Specific Object Detection
Outcome	A two-sentence description of the scene, plus bounding boxes drawn around named objects

Two companion screen recordings (Week6_SPARK_ImageDescriptor.gif, Week6_SPARK_ObjectDetection.gif) show every click in sequence.

Step 1 - Load the Image Descriptor example (Slide 21)

Open SPARK and go to the Gallery. We start with the simplest computer-vision flow: hand an image to a multimodal model and ask it to describe what it sees.

1. On the Gallery tab, find the Image Descriptor card ("Upload an image, and this agent will generate a detailed description of what it sees").
2. Click Load Example. The Canvas opens with three nodes: Upload Image, Image Vision, and Show Description.

The middle node (Image Vision) is the multimodal LLM. The default example uses an OpenAI model, so switch it to the provider whose key you set up.

- Click the Image Vision node, and in Properties set AI Provider to Google Gemini (the Model becomes gemini-2.5-flash).
- In the Prompt box, enter the instruction below.
- Click the Upload Image node, then Upload Image File and choose arsenal-football.jpg.

describe the image in two sentences.

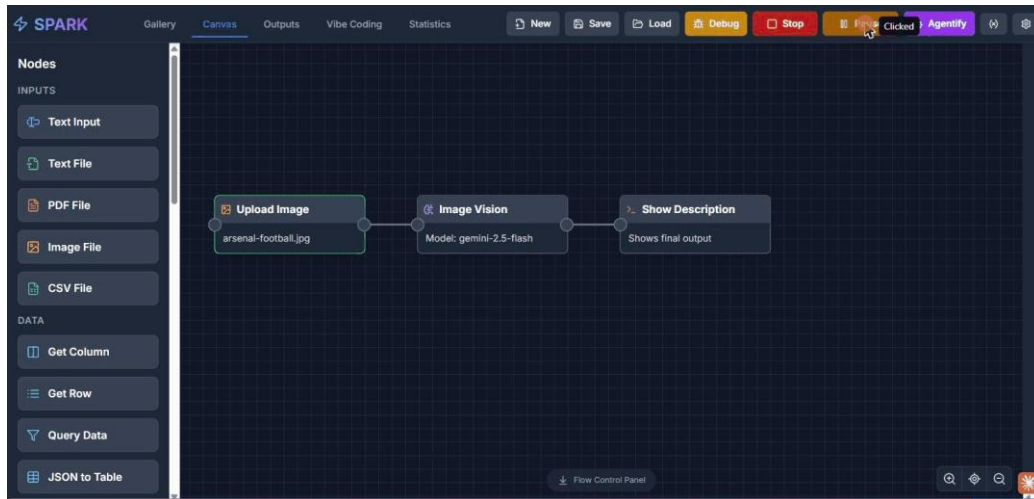


Figure 1 - The Image Describer flow: Upload Image (arsenal-football.jpg) -> Image Vision (gemini-2.5-flash) -> Show Description.

Step 2 - Run and read the description (Slide 21)

Click Run. Gemini looks at the pixels and returns a concise, two-sentence description in the Agent Output dialog.

Notice how much detail a general model picks up unprompted: the two teams and their kit colours, a player heading the ball mid-air, the crowded stadium, even the wet conditions and the goalpost in the foreground. This is the same capability behind image captioning, accessibility alt-text, and content moderation.

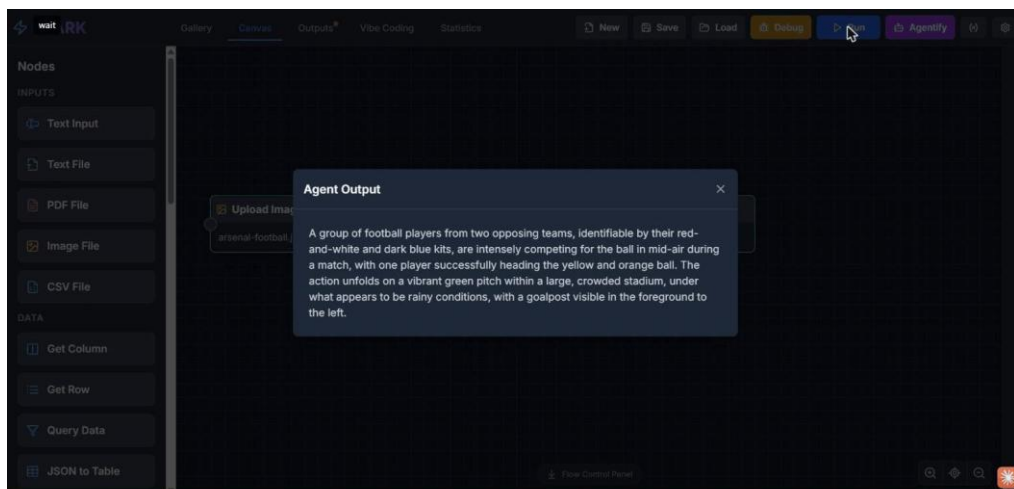


Figure 2 - The Agent Output: a two-sentence description of the football scene.

Tip: change the prompt to steer the output - for example "list every visible brand or sponsor" or "describe only the goalkeeper". The same flow becomes a different tool.

Step 3 - Switch to Object Detection (Slide 22)

Describing an image is one thing; locating a specific object in it is another. The Specific Object Detection example asks the model for the coordinates of an object and draws a box around it.

3. Go back to the Gallery and find the Specific Object Detection card ("Upload an image, describe an object, and have the AI draw a box around it").
4. Click Load Example, then Confirm to clear the canvas.
5. The flow has five nodes: Source Image and Object to Find feed Find Object AI, which feeds Combine for Viewer, then Display Result.
6. Click Find Object AI and set AI Provider to Google Gemini (Model: gemini-2.5-flash).
7. Click Source Image, Upload Image File, and choose arsenal-football.jpg.
8. Click the Object to Find node and change its text to: goal keeper

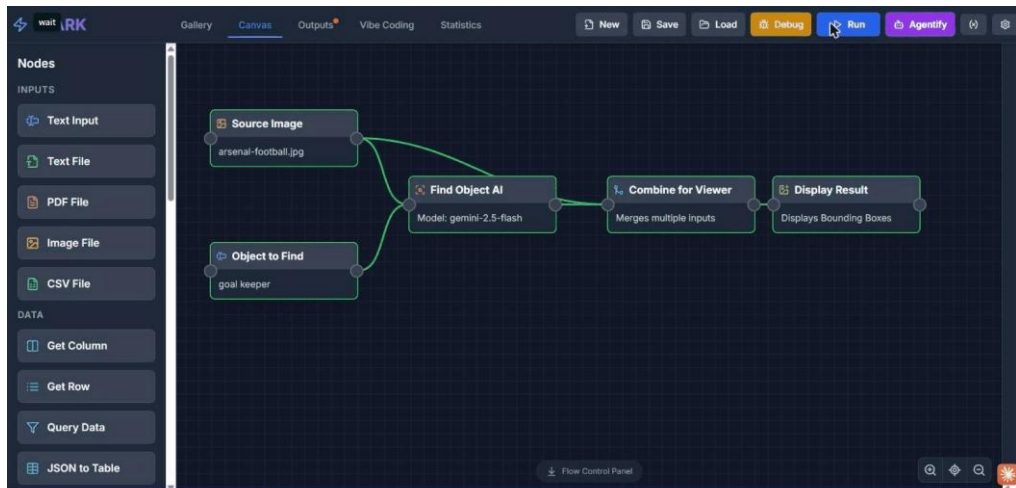


Figure 3 - The object-detection flow: Source Image + Object to Find (goal keeper) -> Find Object AI -> Combine for Viewer -> Display Result.

Step 4 - Run and view the bounding box (Slide 22)

Click Run. When it finishes, click the Display Result node to open the image viewer. The model has located the goalkeeper (in green, defending the goal on the left) and drawn a labelled red box around them.

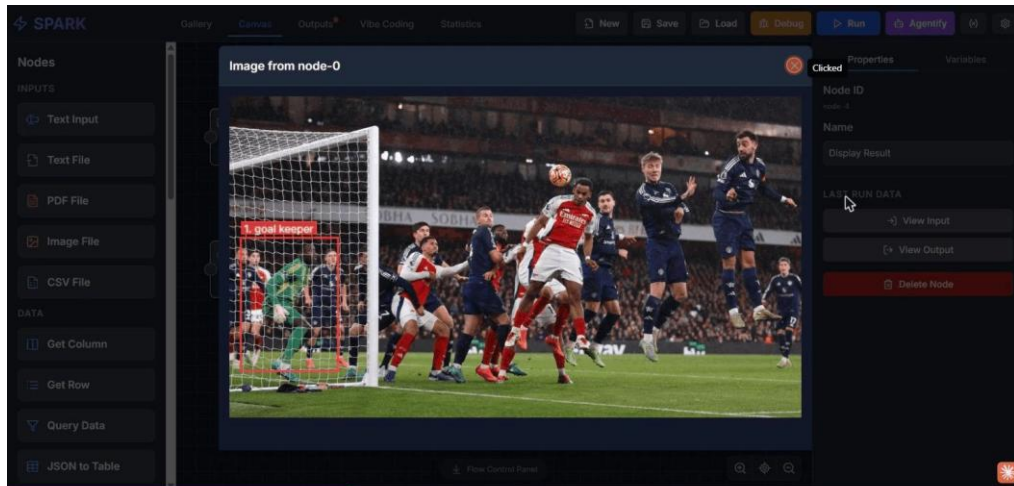


Figure 4 - Display Result: a labelled bounding box around the detected "goal keeper".

Step 5 - Detect multiple objects (Slide 23)

The model can find several instances of the same description at once. Change the Object to Find text and run again.

9. Click the Object to Find node and change its text to: jumping player
10. Click Run, then open Display Result again.

This time the model returns three boxes, each labelled "jumping player", around the players contesting the ball in the air. Try other descriptions too, such as "player that is likely to score".

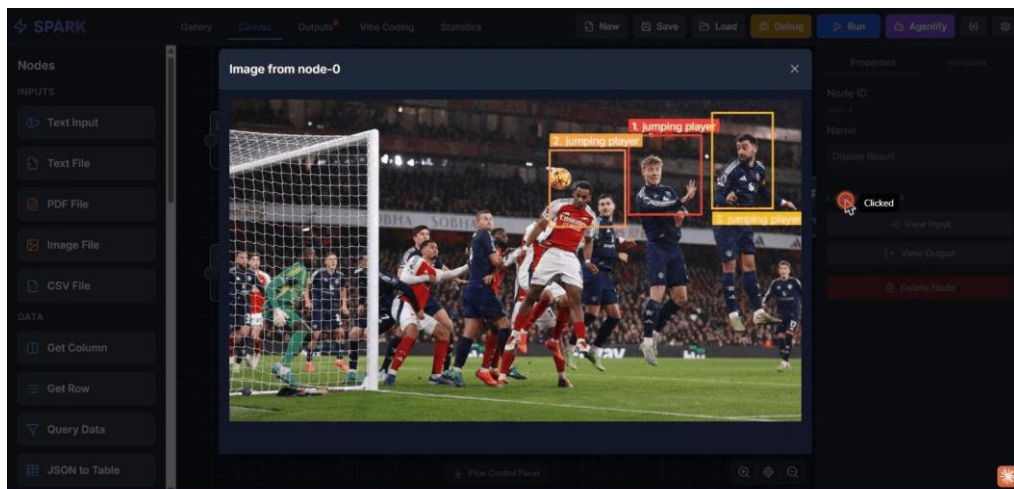


Figure 5 - Multiple detections: three labelled boxes for "jumping player".

Step 6 - Important notes on reliability (Slide 23)

Object detection is a bit unstable in the current SPARK version (the underlying vision APIs have changed), and detection quality depends heavily on the AI model you choose.

You will sometimes hit errors like the one below, where the model returns malformed coordinates instead of valid bounding boxes.

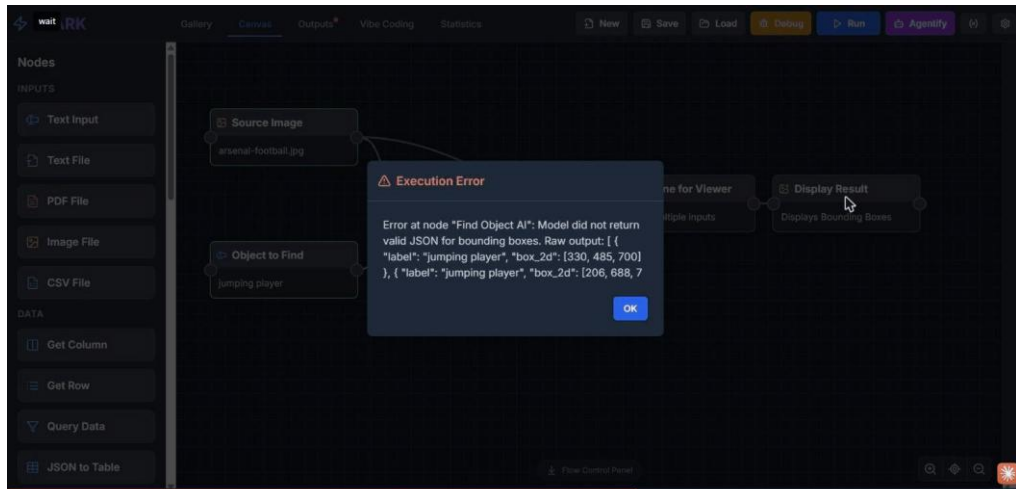


Figure 6 - A real Execution Error - the model did not return valid JSON for the bounding boxes. This is the instability the slide warns about.

If detection fails or the result looks wrong:

- Just run it again - many errors are transient (a malformed response, or a temporary "high demand" message from the API).
- Try a different AI Provider or model on the Find Object AI node; some models return cleaner coordinates than others.
- Simplify the object description - short, concrete phrases ("goal keeper", "jumping player") work better than long ones.
- Remember image description (Steps 1-2) is far more robust than precise object detection - use the right tool for the task.

Good to know

- Set your Gemini key in Settings before running anything; flows fail silently if the provider has no key.
- The same Image Vision node powers many tasks - captioning, alt-text, moderation, visual Q&A - just by changing the prompt.
- Object detection returns coordinates as data, which the Combine for Viewer and Display Result nodes turn into the boxes you see.
- Detection reliability varies by model and can be flaky; re-run or switch models rather than assuming the image is at fault.
- If a node's properties look empty or stale, click away and reselect the node to refresh the panel.